# QUALITY ASSURANCE TESTING DOCUMENTATION

# (QATD)

## UMHackathon 2026

umhackathon@um.edu.my

---

© UMHackathon 2026

Email: umhackathon@um.edu.my

# Table of Content

**Document Control** **3**

**PRELIMINARY ROUND (Test Strategy & Planning)** **3**

1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications 6
6. Test Strategy & Plan Sign-Off 7

*© UMHackathon 2026*

*Email: umhackathon@um.edu.my*

# Document Control

| Field | Detail |
|---|---|
| **System Under Test (SUT)** | UMHackathon 2026 - Team **KetupatPalas** |
| **Team Repo URL** | https://github.com/AnisAtirah/KetupatPalas.git |
| **Project Board URL (Google Docs)** | https://docs.google.com/document/d/1iiD32ZxOZlXgHBWmWmQ40ASnxdK_UDGGBJVekquibGI/edit?usp=sharing |
| **Live Deployment URL** | https://ketupatpalashaidis-production.up.railway.app/ |

## Objective:

The objective of this QATD is to ensure that HaiDIS delivers reliable, accurate, and efficient performance by systematically validating its core features, including real-time data processing, AI-driven recommendations, and what-if simulations. It also ensures seamless integration between frontend, backend, and external AI services, while maintaining system stability, performance, and data integrity under expected usage conditions.

# PRELIMINARY ROUND (Test Strategy & Planning)

## 1. Scope & Requirements Traceability

### 1.1 In-Scope Core Features

- **AI-powered Decision Recommendation:** The system generates recommendations using predefined logic and simulated AI reasoning based on input data.
- **Interactive Dashboard:** The system implements an interactive and visually engaging interface to display clear visualization of hospital information.
- **What-If Simulation Feature:** The simulation model is used for predicting impact of resource modifications to assist in decision making.
- **Cost vs Efficiency Analysis:** The system shows trade-offs between operational cost and efficiency for better understanding of the purpose behind recommendations given.
- **Manual Data Input for Hospital Resources:** The system relies on manual user input for data such as number of patients, staff and beds as the basis for analysis and simulation.

### 1.2 Out-of-Scope

- **Real-time Hospital Data Integration:** The system will not connect to live hospital databases or Electronic Health Records (EHR). It relies solely on manual user (hospital management staff) inputs for analysis.
- **Live Shift Scheduling & Staff Communication:** While the system provides general shift planning suggestions, it does not include a functional module for staff to clock in/out or receive automated assignments.
- **Predictive Alert System:** Automated "Smart Alerts" for future overstock or low demand risks are not included in this version.
- **User Authentication & Multi-Role Access:** The current scope does not support secure login, user profiles, or different access levels (e.g., separate views for doctors vs. administrators).
- **Financial Transaction Processing:** The "Cost vs Efficiency" feature is for estimation and simulation purposes only; it does not handle actual hospital billing or payment processing.
- **Mobile Application:** This project is strictly a web-based platform and does not include a native mobile app (iOS/Android).

---

## 2. Risk Assessment & Mitigation Strategy

| Technical Risk | Likelihood (1–5) | Severity (1–5) | Risk Score (L×S) | Mitigation Strategy | Testing Approach |
|---|---|---|---|---|---|
| Real-time data not updating or delayed | 4 | 4 | 16 | Implement auto-refresh intervals and fallback caching system to ensure data availability | Simulate high traffic and network delay; monitor dashboard update intervals |
| AI generates similar or generic recommendations across different hospital scenarios | 3 | 4 | 12 | Use contextual prompting with real-time hospital inputs (e.g. patients, beds, staff) to ensure dynamic and | Test multiple input scenarios with different data sets and compare outputs for uniqueness and relevance |

© UMHackathon 2026

Email: umhackathon@um.edu.my

| | | | | | |
|---|---|---|---|---|---|
| | | | | scenario-specific AI outputs | |
| What-If Simulator producing unrealistic results | 3 | 4 | 12 | Set logical boundaries and constraints for simulation variables (e.g. max staff, bed limits) | Perform boundary testing and compare outputs with expected logical outcomes |
| UI layout breaks during simulation or when displaying large AI-generated outputs | 2 | 4 | 8 | Implement responsive grid system with fixed containers for dashboard and simulation outputs to maintain layout stability | Perform UI stress testing with long AI responses and different screen sizes to ensure layout consistency |
| Incorrect cost vs efficiency calculations | 3 | 4 | 12 | Implement validation formulas and cross-verification logic | Test with known sample data and verify calculation accuracy |
| AI fails to interpret flowcharts or system logic correctly | 3 | 3 | 9 | Convert all system logic, architecture, and data flows into structured text/code instead of relying on AI interpretation; implement fallback error handling | Input invalid or incomplete data to trigger fallback responses and verify system stability and error messaging |

**Risk Assessment Scoring Criteria:**

| Likelihood (1-5) | | Severity (1-5) | |
|---|---|---|---|
| 1 | Rare | 1 | Impact is Negligible |
| 2 | Unlikely | 2 | Impact is Minor |

© UMHackathon 2026

Email: umhackathon@um.edu.my

| | | | |
|---|---|---|---|
| 3 | Possible | 3 | Moderate Impact |
| 4 | Likely | 4 | Major Impact |
| 5 | Almost Certain | 5 | Critical Failure of the system |

**Risk Score = Likelihood × Severity**

**Risk Level Reference:**

| Risk Score | Risk Level | Recommended Action |
|---|---|---|
| 1 – 5 | Low | Monitor only. Acceptable risks. |
| 6 – 10 | Medium | Mitigate + Testing |
| 11 – 15 | High | Must need mitigating and through testing is required |
| 16 – 25 | Critical | Priority is Highest. Need extensive level of testing. |

## 3. Test Environment & Execution Strategy

- **Unit Test**
  - Scope: Core logic functions within HaiDIS's Python modules, specifically calculate_cost() and calculate_waiting_time() in costvsefficiency.py, build_prompt_payload() and _rule_based_suggestions() in analytics.py, and the fallback logic in ai_service.py.
  - Execution: Tests are written using PyTest and executed locally during development. Each function is tested in isolation before being integrated with other modules.
  - Isolation: External dependencies such as the hospital management database and the GLM API are mocked during unit testing. For example, when testing _rule_based_suggestions(), hardcoded zone values (Red, Yellow, Green) are passed directly without loading real CSV data. When testing ai_service.py fallback logic, the GLM API call is mocked to simulate both a successful response and a failure scenario.
  - Pass Condition: A unit test is considered passing when it satisfies all of the following: correct output for happy path inputs (e.g. valid zone returns three suggestions), correct handling of negative cases (e.g. missing API key triggers fallback), and correct handling of edge cases (e.g. zero available beds, maximum patient load, empty suggestions list returned by GLM).

© UMHackathon 2026
Email: umhackathon@um.edu.my

- **Integration Test**
  - Scope: End-to-end flow from the Flask API endpoints to the Python service modules and back to the frontend. Key integration paths tested include:
    - /api/recommendations → ai_service.py → analytics.py → GLM API
    - /api/simulate → costvsefficiency.py → JSON response to app.js
    - /api/cost-efficiency → analytics.py → build_cost_efficiency_comparison() → JSON response
    - /api/context → analytics.py → build_recommendation_context() → JSON response

  - Execution: Integration testing is performed after unit tests for all individual modules pass. Tests are run on a local Flask development server (127.0.0.1:5000) using real HTTP requests to verify that all components work correctly when connected together.

  - Workflow: No mocking is used at this stage. Real calls are made to the ILMU GLM-5.1 API using the configured ILMU_API_KEY, and real data is loaded from the hospital management database to verify the full data flow. For the simulation endpoint, multiple consecutive calls are made to verify that the global state in costvsefficiency.py updates correctly across ticks and that the frontend reflects the correct sequence of changes.

  - Pass Condition: Integration tests pass when all four Flask endpoints return HTTP status 200, the response body is valid JSON, and the data fields match the expected schema. Specifically, /api/recommendations must return exactly three suggestion objects each containing title, explanation, and priority. /api/simulate must return cost_change, waiting_time_change, current_cost, current_waiting_time, patients_served, interpretation, insight, and a state object containing doctors, nurses, icu_patients, ward_patients, and er_patients.

- **Test Environment (CI/CD Practice):**
  - Local Testing: All unit and integration tests are first executed manually on localhost during development. Developers run pytest locally before committing any changes to the repository.

  - Staging/CI: GitHub Actions is configured to trigger automated PyTest runs on every push to any branch. This catches regressions early before code is reviewed or merged.

  - CI/CD Automated Pipeline: On pull requests targeting the main branch, GitHub Actions runs the full test suite including unit tests, integration tests, and a basic smoke test against the deployed Vercel staging URL to confirm the

© UMHackathon 2026

application loads and all four API endpoints respond correctly including the simulation endpoint which must return a valid tick response on first call.

- **Regression Testing & Pass/Fail Rules:**
  - Execution Phase: Regression testing is performed each time a new branch is merged into main. All four core features of HaiDIS are re-tested:- the situation dashboard (/api/context), the AI recommendation engine (/api/recommendations), the cost efficiency comparison (/api/cost-efficiency), and the what-if simulation (/api/simulate).

  - Pass/Fail Condition: A test case is marked as passed only when the actual output exactly matches the expected output defined in the test specification. Any mismatch including incorrect field values, wrong HTTP status codes, missing JSON keys, simulation state not updating correctly across ticks, or unexpected fallback activation is marked as failed and logged in the Defect Log with the test case ID, expected outcome, actual outcome, and severity level.

  - Continuation Rule: Regression testing proceeds in order of feature criticality:- situation dashboard first, then AI recommendations, then cost efficiency comparison, then what-if simulation. If the situation dashboard tests do not meet the passing threshold, testing halts and the defect must be resolved before proceeding to the next feature. This prevents cascading failures from masking the root cause of an issue.

- **Test Data Strategy:**
  - Automated-Live Database Simulation: To simulate real-world hospital conditions, the system automatically inserts randomised data into the hospital management database every 5 minutes. This simulated data reflects realistic fluctuations in patient numbers across the pediatric departments, as well as changes in doctor availability, nurse availability, and bed occupancy. This approach eliminates the need for manually crafted test snapshots and ensures that HaiDIS is continuously tested against dynamic, evolving data that mirrors actual hospital operations.

  - What the simulation covers:
    - Increasing and decreasing patient counts
    - Fluctuations in available doctors and nurses on duty
    - Changes in bed occupancy that trigger zone transitions between Green, Yellow, and Red
    - Edge cases that naturally emerge from the randomisation, such as hitting minimum staff thresholds

© UMHackathon 2026
Email: umhackathon@um.edu.my

- **Passing Rate Threshold:**
    - A minimum of 85% of all executed test cases must pass for a build to be considered stable and eligible for merging into main. For critical test cases specifically those covering the GLM API response validation, fallback activation, zone classification logic, and simulation state boundary enforcement (e.g. doctor count never drops below 1, ICU patients never exceed 10), a 100% pass rate is mandatory with no exceptions, as failure in these areas directly impacts the reliability of both the AI recommendations and the simulation outputs presented to hospital staff. Any build that fails critical tests is automatically blocked from merging regardless of the overall pass rate.

## 4. CI/CD Release Thresholds & Automation Gates

This section defines the quality thresholds and success criteria required before code changes can be merged into the main branch or deployed to production. If any threshold is not met, the pipeline blocks progression to ensure system stability and reliability.

### 4.1 Integration Thresholds (Merging to Main)

The main branch serves as the **stable source of truth**, so all changes must pass strict validation checks before merging.

| **Checks** | **Requirements** | **Project Result** | **Status** |
|---|---|---|---|
| Build | Zero Build Error | No crash and API works | Passed |
| UI Functionality | 100% Passing Rate | Core dashboard and simulator is working | Passed |
| Unit Tests | 100% Passing Rate | Passed 7 test cases (100% pass rate) | Passed |
| Code Quality | Zero Linting Error | Minor formatting and unused variable warning | Partial |
| Test Coverage | Minimum 85% Coverage | 51%  coverage | Failed |

© UMHackathon 2026
Email: umhackathon@um.edu.my

### 4.2 Deployment Thresholds (Pushing to Production)

| Checks | Requirements | Project | Pass/Failed |
|---|---|---|---|
| Regression Test | Minimum 90% | 100% with 7 test | Passed |
| AI Output Pass Rate | Minimum of 80% of documented prompt | 78% | Failed |
| Critical Bugs | Zero P0/01 Bugs | 0 | Passed |
| API Performance | Response Time < 800ms | 488 ms | Passed |
| Security | API keys are not exposed | Clean | Passed |

## 5. Test Case Specifications

| Test Case ID | Test Type & Mapped Feature | Test Description | Test Steps | Expected Result | Actual Result |
|---|---|---|---|---|---|
| TC-01 | **Happy Case (Entire Flow):** Resource Allocation Flow | Verify that a user can input hospital data and receive a valid AI recommendation with a cost-impact analysis. | 1. Navigate to Simulator.<br><br>2. Input:,4 Doctors, 10 Nurses, 20 Beds.<br><br>3. Click "Analyze". | AI returns a recommendation (e.g., "Add 2 Nurses") with a specific impact percentage (e.g., "Wait time -20%"). | Successfully generated. The AI suggested Add 1-2 more doctors. Since doctors have a higher workload (6.67 vs 6.15) and wait times are still long, doctors are likely the bottleneck .Current changes add RM6,200. Hiring more doctors will increase costs |

© UMHackathon 2026

Email: umhackathon@um.edu.my

| | | | | | |
|---|---|---|---|---|---|
| | | | | | slightly, but it should help reduce the long wait time.<br><br>**Status: Passed** |
| TC-02 | **Specific Case (Negative):** Invalid Input Handling | Verify that the system prevents users from entering negative values in the What-If Simulator using input constraints, ensuring only valid non-negative numbers are accepted. | 1. Navigate to Simulator.<br><br>2. Attempt to input Doctor: -10, Nurse: -5, Beds -3. | System does not allow negative value to be entered as the input fields restrict value to minimum = 0. User can only input valid non-negative integers. | Negative value cannot be entered into input fields and automatically stays at 0 when clicked lower.<br><br>**Status: Passed** |
| TC-03 | **NFR (Performance):** API Latency (Architecture) | Verify the GLM API returns a recommendation within an acceptable time frame for a smooth user experience. | 1. Trigger a standard recommendation request.<br><br>2. Measure time from "Click" to "Display". | The system returns AI recommendations within an acceptable latency range of ≤ 2 minutes, and UI remains responsive. | The fastest response observed is approximately 2 minutes under stable gateway conditions. The system successfully returned a recommendation, and UI remained stable.<br><br>**Status: Passed** |
| TC-04 | **NFR (Reliability):** AI Recommendation on Dashboard | Verify that the system can handle multiple consecutive AI recommendation requests without crashing or degrading performance, | 1. Navigate to Dashboard<br><br>2. Click the "Generate +" button repeatedly (e.g., 5–10 times) | System handles repeated request without crashing and each request return valid recommendation. | Multiple AI request were successfully processed and recommendation were displayed correctly each time<br><br>**Status: Passed** |

© UMHackathon 2026

Email: umhackathon@um.edu.my

| | | | | | |
|---|---|---|---|---|---|
| | | ensuring stable dashboard operation. | 3. Observe | | |

© UMHackathon 2026

Email: umhackathon@um.edu.my

## 6. AI Output & Boundary Testing

This section validates that HaiDIS's GLM integration produces accurate, well-structured, and contextually appropriate outputs under both normal and abnormal conditions.

### 6.1 Prompt/Response Test Pairs

| Test ID | Prompt Input | Expected Output (Acceptance Criteria) | Actual AI Output | Status |
|---|---|---|---|---|
| AI-01 | High load scenario (e.g., 19 patients, 4 doctors, 7 nurses) | Should provide a clear impact on system performance, including logical reasoning related to workload or patient flow. | **High Hire** Additional Doctors<br><br>With 19 patients and only 4 doctors, the patient-to-doctor ratio of approximately 4.75:1 is contributing to increased waiting times. Adding 1-2 more doctors would help reduce the current wait time of 20.9 minutes and improve patient throughput.<br><br>**Medium** Optimize Nurse Task Delegation<br><br>With 7 nurses available, there is an opportunity to better utilize nursing staff for routine assessments, patient triage, and administrative tasks. This would free up doctors to focus on clinical duties, improving overall efficiency and helping control cost growth.<br><br>**High** Implement a Patient Triage System<br><br>A structured triage protocol would prioritize patients based on urgency, ensuring critical cases are seen promptly while reducing unnecessary wait times. This can improve patient | Pass |

© UMHackathon 2026

Email: umhackathon@um.edu.my

| | | | | |
|---|---|---|---|---|
| | | | flow without requiring additional hiring, helping address both the wait time and cost increase simultaneously. | |
| AI-02 | The user runs What-If simulation with additional doctors, nurses, or beds | AI output must reflect updated simulation results and provide relevant justification based on workload, wait time, or patient flow. | Performance Analysis:<br><br>The hospital is under moderate pressure with a high wait time of 73.5 minutes for 80 patients. This happens because the current doctor and nurse workloads are stretched, creating a processing bottleneck that delays patient care.<br><br>Recommendation:<br><br>Add one more doctor and nurse to the shift. This will lower the patient-to-staff ratio, speed up the processing time, and directly reduce the long wait times.<br><br>Cost Impact:<br><br>The added cost of RM 15240 is reasonable. Reducing the excessive wait time prevents staff burnout and improves patient safety and satisfaction, which provides long-term value that justifies the staffing expense. | Pass |

### 6.2. Oversized/Larger Input Test

| **Fields** | **Details** |
|---|---|
| **Maximum Input Size** | Numeric values only (non-text input enforced by system), |

© UMHackathon 2026
Email: umhackathon@um.edu.my

| | |
|---|---|
| **Input used while testing** | Attempted to input a 10,000-character string (random text) into numeric fields. |
| **Expected Behavior** | The system should prevent non-numeric input and reject oversized text data. |
| **Actual Behavior** | The UI did not allow any text input. Only numeric values could be entered into the fields. The oversized string could not be typed or submitted. |
| **Status** | Passed |

### 6.3. Adversarial/Edge Prompt Test

In this system, the prompt is not entered by the user. Instead, it is automatically generated by the backend using simulation results (e.g., wait time, doctor load, nurse load, cost). Therefore, adversarial testing focuses on how the system handles unexpected or problematic AI responses, rather than malformed user input.

| Field | Details |
|---|---|
| **Test ID** | AI-EDGE-01 |
| **Test Type** | Malformed / Unexpected AI Response. |
| **Input Condition** | System-generated prompt based on simulation data (e.g., high wait time, high workload). |
| **Adversarial Scenario** | AI returns incomplete or poorly structured output (e.g., missing "Recommendation" or mixed paragraphs). |
| **Expected Behavior** | System should detect invalid format and prevent incorrect display. |
| **Actual Behavior** | The system checks the AI output structure. If parsing fails, it returns an error response instead of displaying incorrect information. |
| **System Handling** | Error is returned: **"AI recommendation failed. Please try again later."** |
| **Status** | Passed |

© UMHackathon 2026
Email: umhackathon@um.edu.my

### 6.4. Hallucinating Handling

Hallucination in the context of HaiDIS refers to GLM generating content that is factually inconsistent with the provided data, structurally invalid, or outside the defined scope. Since HaiDIS has two separate AI features with different output formats, hallucination mitigation is handled differently for each.

**Feature 1: Dashboard AI Recommendations**

- **Closed data context**
  The most significant hallucination mitigation for this feature is architectural. The model is never exposed to open-ended user queries or external knowledge. Every inference call is grounded strictly in the pre-aggregated JSON payload produced by analytics.py, which contains only verified metrics from the hospital management database. The system prompt explicitly instructs the model to base its response solely on the provided data summary and not to introduce outside information.

- **Strict output schema enforcement**
  The API call sets response_format: json_object, forcing GLM to return valid JSON only. This eliminates a common hallucination pattern where models produce free-form narrative responses that mix fabricated context with factual data.

- **Structural validation**
  After the response is received, ai_service.py parses and validates the output before it is passed to the frontend. Specifically, the system checks that:

  - The response can be successfully parsed by json.loads()
  - The suggestions key exists in the parsed response
  - The value of suggestions is a non-empty list

  If any of these checks fail, a ValueError is raised immediately and the fallback mechanism activates.

- **Rule-based fallback as a safety net**
  When structural validation fails which can be caused by a hallucinated response that breaks the expected schema. HaiDIS automatically returns pre-computed rule-based suggestions from analytics.py instead. This ensures the dashboard always displays grounded, deterministic recommendations even when the model output is unusable.

**Feature 2: What-If AI Recommendation**

- **Numeric-only inputs**
  The model is never exposed to free-text user input. The user only provides numeric values (additional doctors, nurses, and beds) which are pre-processed through the

© UMHackathon 2026

simulate() function before reaching GLM. The model only ever sees the computed simulation result, not raw user input.

- **Prompt scope restriction**
  The prompt explicitly instructs GLM to respond in exactly three plain text sections (Analysis, Recommendation, and Cost Impact) without JSON or markdown. This constrains the model's output to a narrow, well-defined scope and reduces the risk of the model introducing irrelevant or fabricated content.

- **Line-by-line section parsing**
  After the response is received, ask_ai_for_simulation() parses the plain text response line by line, identifying each section by its keyword prefix. This allows the system to detect whether each expected section is present and non-empty.

- **ValueError on invalid format**
  If any of the three required sections (Analysis, Recommendation, or Cost Impact) is found to be empty or missing after parsing, a ValueError is raised. The system then returns an HTTP 500 error along with the raw simulation result, so the user still has access to the quantitative output even without the AI narrative.

**Important Note:**
Using Agile principle, these testing will take place iteratively over the period of development, not limited to at the end.

*© UMHackathon 2026*

*Email: umhackathon@um.edu.my*